

TECH

TAKE-HOME ЗАДАЧИ

TAKE-HOME

TAKE-HOME

25 ЗАДАЧ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК общая практика индустрии

20 ВОПРОСОВ 7 ДНЕЙ

День 1 тяжёлый	MARKET WATCH DASHBOARD Q1 · Q2 · Q3	POSITIONS	TRADING D	MARKET WATCH DASHBOARD	POSITIONS	TRADING D
День 2 тяжёлый	ORDER BOOK ADMIN TABLE Q4 · Q5 · Q6	NOTIFICATION CENTER	A	ORDER BOOK ADMIN TABLE	NOTIFICATION CENTER	A
День 3 тяжёлый	PORTAL AUTH D REMOTE Q7 · Q8 · Q9	PWA OFFLINE	FEDERATE	PORTAL AUTH D REMOTE	PWA OFFLINE	FEDERATE
День 4 средний	MINI DESIGN SYSTEM ATE FORM BUILDER Q10 · Q11 · Q12	DASHBOARD URL ST		MINI DESIGN SYSTEM ATE FORM BUILDER	DASHBOARD URL ST	
День 5 средний	FLAGS UI ERROR DASH Q13 · Q14 · Q15 · Q16 · Q17	A/B UI PERF PLAYGROUND	ANALYTICS DEBUGG	FLAGS UI ERROR DASH	A/B UI PERF PLAYGROUND	ANALYTICS DEBUGG
День 6 тяжёлый	JS→TS MIGRATION QUERY OPTIMISTIC CLIENT Q18 · Q19 · Q20 · Q21 · Q22	REFACTOR LEGACY A11Y PACKAGE	WS	JS→TS MIGRATION QUERY OPTIMISTIC CLIENT	REFACTOR LEGACY A11Y PACKAGE	WS
День 7 тяжёлый	VIRTUALIZED TABLE PELINE Q23 · Q24 · Q25	MF SHELL	CI PI	VIRTUALIZED TABLE PELINE	MF SHELL	CI PI

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

MARKET WATCH

POSITIONS

TRADING DASHBOARD

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MARKET WATCH POSITIONS TRADING DASHBOARD

ВОПРОСЫ ДНЯ

Q01 Market Watch widget с real-time котировками.

Q02 Positions Table с search/sort/filter/pagination.

Q03 Trading dashboard с WebSocket feed.

Market Watch widget с real-time котировками.

ОПИСАНИЕ

Виджет, отображающий список инструментов с обновляемой ценой. Architecture: WS-feed → батчинг → store → виртуализация → flash-эффект на изменении.

КАК РАБОТАЕТ

- 01 WS-подключение с reconnect/backoff.
- 02 Батч-апдейтер (накопить 50мс → flush).
- 03 useSyncExternalStore + селектор по символу.
- 04 CSS-flash через transform/opacity (не layout-shift).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading-platforms.
- 02 Crypto-tickers.

ПОДВОДНЫЕ КАМНИ

- 01 setState на каждый tick → render storm.
- 02 Layout-shift при flash → CLS.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tick-to-paint?
- 02 Чем виртуализация помогает?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/useSyncExternalStore>

ВОПРОС Q02

Positions Table с search/sort/filter/pagination.**ОПИСАНИЕ**

Таблица позиций с фильтрами/сортировкой/пагинацией. TanStack Table для логики, server-side для очень больших объемов, URL-state для shareable-ссылок.

КАК РАБОТАЕТ

- 01 TanStack Table + Virtual.
- 02 URL: ?sort=&filter=&page=.
- 03 Серверная сортировка/фильтрация.
- 04 Skeleton при loading.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Admin-панели.
- 02 Reporting.

ПОДВОДНЫЕ КАМНИ

- 01 Без virtualization – лагает.
- 02 Без URL-state – refresh теряет контекст.

МОГУТ ДОСПРОСИТЬ

- 01 Чем server-side от client-side отличается?
- 02 Где хранить sort?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/table/latest>

Trading dashboard с WebSocket feed.

ОПИСАНИЕ

Дашборд из независимых виджетов, каждый со своим WS-каналом. ErrorBoundary вокруг каждого, общий connection-manager, batched updates, virtualization.

КАК РАБОТАЕТ

- 01 Connection manager – singleton WS.
- 02 Multiplexing channels через subscribe/unsubscribe.
- 03 ErrorBoundary per widget.
- 04 Layout grid + responsive.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading-applications.
- 02 Live monitoring.

ПОДВОДНЫЕ КАМНИ

- 01 Один WS на виджет – N подключений.
- 02 Без ErrorBoundary один баг ломает всё.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое connection multiplexing?
- 02 Как тестировать WS-логику?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

ДЕНЬ 2 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

ORDER BOOK

NOTIFICATION CENTER

ADMIN TABLE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ORDER BOOK NOTIFICATION CENTER ADMIN TABLE

ВОПРОСЫ ДНЯ

- Q04 Order Book UI с high-frequency updates.
- Q05 Notification center с read/unread state.
- Q06 Admin table с permissions и audit log.

Order Book UI с high-frequency updates.

ОПИСАНИЕ

Список bids/asks, обновляющийся 100+ раз/сек. Snapshot + incremental updates, дифф по цене, виртуализация, минимальный re-render. Иногда canvas/WebGL.

КАК РАБОТАЕТ

- 01 Snapshot + delta-updates.
- 02 Map<price, level> для O(1) update.
- 03 Virtualization (react-virtual).
- 04 Иногда canvas для совсем тяжёлых случаев.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Crypto / forex.
- 02 L2 market data.

ПОДВОДНЫЕ КАМНИ

- 01 DOM-manipulation на 1000+ tick/sec – kill UI.
- 02 Без диффа – всё пересоздаётся.

МОГУТ ДОСПРОСИТЬ

- 01 Когда canvas лучше DOM?
- 02 Что такое L2/L3 data?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/canvas-performance>

ВОПРОС Q05

Notification center с read/unread state.

ОПИСАНИЕ

Список уведомлений с фильтром read/unread, пагинация, real-time push, mark-as-read с optimistic update, badge с count.

КАК РАБОТАЕТ

- 01 WS push для новых.
- 02 Optimistic mark-as-read.
- 03 Persistent state через react-query / Zustand.
- 04 Badge с unread-count в header.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 B2C apps.
- 02 SaaS-инструменты.

ПОДВОДНЫЕ КАМНИ

- 01 Без optimistic – laggy UI.
- 02 Race race race на mark-as-read.

МОГУТ ДОСПРОСИТЬ

- 01 Чем optimistic update полезен?
- 02 Где persist read state?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

Admin table с permissions и audit log.

ОПИСАНИЕ

Таблица user-records с inline-actions (edit/delete/disable), permission-based visibility, audit-log при изменениях.

КАК РАБОТАЕТ

- 01 RBAC – Can-component для actions.
- 02 Inline-edit с optimistic save.
- 03 Audit-log entry на каждое изменение.
- 04 Bulk actions с confirm-modal.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 B2B admin tools.
- 02 User management.

ПОДВОДНЫЕ КАМНИ

- 01 Frontend-only RBAC = security hole.
- 02 Bulk actions без confirm – катастрофа.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RBAC?
- 02 Где хранить audit-log?

ПОЛНАЯ СТАТЬЯ

https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

3 ВОПРОСА

PORTAL AUTH**PWA OFFLINE****FEDERATED REMOTE**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PORTAL AUTH PWA OFFLINE FEDERATED REMOTE

ВОПРОСЫ ДНЯ

Q07 Internal portal module с auth boundary.

Q08 PWA dashboard с offline fallback.

Q09 Federated remote app, подключаемый к shell.

Internal portal module с auth boundary.

ОПИСАНИЕ

Модуль внутреннего портала с auth-проверкой, redirect на login, refresh-token flow, protected routes, layout с навигацией.

КАК РАБОТАЕТ

- 01 AuthProvider в корне.
- 02 ProtectedRoute с redirect.
- 03 Refresh-token middleware.
- 04 Auto-logout при 401.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Internal SaaS.
- 02 Employee portals.

ПОДВОДНЫЕ КАМНИ

- 01 Token в localStorage = XSS-риск.
- 02 Refresh race на параллельных 401.

МОГУТ ДОСПРОСИТЬ

- 01 Где хранить token?
- 02 Чем httpOnly cookie от localStorage отличается?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-top-ten/>

ВОПРОС Q08

PWA dashboard с offline fallback.**ОПИСАНИЕ**

Дашборд с offline-режимом: Service Worker кэширует статику и API-ответы, при offline показывает cached + banner, sync при появлении сети.

КАК РАБОТАЕТ

- 01 Service Worker register.
- 02 Cache API: precache static, runtime API.
- 03 Background Sync для отложенной отправки.
- 04 Offline banner.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile-friendly dashboards.
- 02 Field-service tools.

ПОДВОДНЫЕ КАМНИ

- 01 Stale без указания → confusion.
- 02 POST офлайн → нужен Background Sync.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stale-while-revalidate?
- 02 Как тестировать offline?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/explore/progressive-web-apps>

Federated remote app, подключаемый к shell.

ОПИСАНИЕ

Module Federation: shell + remote, shared deps (React/router), runtime composition, error-boundary вокруг remote, fallback при загрузке.

КАК РАБОТАЕТ

- 01 webpack ModuleFederationPlugin.
- 02 shared singleton: react, react-dom.
- 03 ErrorBoundary вокруг remote-mount.
- 04 Versioned releases.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team SPA.
- 02 Plugin-системы.

ПОДВОДНЫЕ КАМНИ

- 01 Version skew → cannot use hooks outside Component.
- 02 Без EB одна ошибка убивает shell.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое singleton в MF?
- 02 Как обрабатывать chunk-load error?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

MINI DESIGN SYSTEM

DASHBOARD URL STATE

FORM BUILDER

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MINI DESIGN SYSTEM DASHBOARD URL STATE FORM BUILDER

ВОПРОСЫ ДНЯ

Q10 Mini design system с Button/Input/Modal/Table.

Q11 Dashboard с charts, filters и URL state.

Q12 Form builder с validation schema.

Mini design system с Button/Input/Modal/Table.

ОПИСАНИЕ

Базовый UI-кит: токены (CSS vars), 4 компонента с consistent API, headless-pattern, accessibility, Storybook docs.

КАК РАБОТАЕТ

- 01 CSS-variables tokens.
- 02 Compound + polymorphic API.
- 03 Storybook playground.
- 04 Tests: unit + ally + visual.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DS pilot.
- 02 B2B-инструменты.

ПОДВОДНЫЕ КАМНИ

- 01 20+ props на Button – слишком жирно.
- 02 Без ally – недоступно.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое headless component?
- 02 Чем visual regression полезен?

ПОЛНАЯ СТАТЬЯ

<https://www.designsystems.com/>

ВОПРОС Q11

Dashboard с charts, filters и URL state.

ОПИСАНИЕ

Чарты с фильтрами, фильтры в URL для shareable-ссылок, react-query для data, useDeferredValue для тяжёлых перерисовок.

КАК РАБОТАЕТ

- 01 URL-state через router (?from=&to=&metric=).
- 02 react-query с queryKey включающим filters.
- 03 useDeferredValue для charts.
- 04 Skeleton + ErrorBoundary.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Analytics dashboards.
- 02 Reports.

ПОДВОДНЫЕ КАМНИ

- 01 Без URL – refresh теряет контекст.
- 02 Без deferred – лагает при typing.

МОГУТ ДОСПРОСИТЬ

- 01 Чем queryKey важен?
- 02 Что такое useDeferredValue?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest>

Form builder c validation schema.

ОПИСАНИЕ

Конструктор форм: drag-drop полей, конфиг сохраняется как JSON, runtime – рендер по конфигу, валидация через Zod-схему, генерируемую из конфига.

КАК РАБОТАЕТ

- 01 JSON-schema конфиг.
- 02 Field-renderer по type.
- 03 Build Zod-схема из конфига.
- 04 Preview-режим с валидацией.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 No-code форм-builders.
- 02 Survey tools.

ПОДВОДНЫЕ КАМНИ

- 01 JSON-схема без миграций – break при изменении формата.
- 02 Динамическая валидация без типов – баги.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое JSON Schema?
- 02 Как сгенерировать Zod из JSON?

ПОЛНАЯ СТАТЬЯ

<https://json-schema.org/>

ДЕНЬ 5

НАГРУЗКА

СРЕДНИЙ

5 ВОПРОСОВ

FLAGS UI**A/B UI****ANALYTICS DEBUGGER****ERROR DASH**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FLAGS UI

A/B UI

ANALYTICS DEBUGGER

ERROR DASH

PERF PLAYGROUND

ВОПРОСЫ ДНЯ

Q13 Feature flag admin UI.

Q14 A/B test assignment UI.

Q15 Analytics event debugger.

Q16 Error monitoring dashboard.

Q17 Frontend performance playground.

Feature flag admin UI.

ОПИСАНИЕ

Список флагов, фильтры по env/owner, toggle (с confirm), targeting rules, history. Audit-log изменений. Permissions.

КАК РАБОТАЕТ

- 01 Table с inline toggle.
- 02 Confirm-modal перед prod-toggle.
- 03 Audit log row.
- 04 RBAC для prod-changes.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 LaunchDarkly-like internal.
- 02 Custom flag-systems.

ПОДВОДНЫЕ КАМНИ

- 01 Toggle без confirm = инцидент.
- 02 Без audit – кто и когда?

МОГУТ ДОСПРОСИТЬ

- 01 Что такое targeting rules?
- 02 Как rollback toggle?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/>

ВОПРОС Q14

A/B test assignment UI.**ОПИСАНИЕ**

Список экспериментов, сегменты пользователей, конверсии, statistical significance. Часто строится поверх существующего flag-сервиса.

КАК РАБОТАЕТ

- 01 Experiment list с status.
- 02 Segments / variants.
- 03 Conversion metrics + significance.
- 04 Stop-button для kill-switch.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Product analytics.
- 02 Growth-команды.

ПОДВОДНЫЕ КАМНИ

- 01 Без significance – false-positive.
- 02 Малый sample → невалидно.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое statistical significance?
- 02 Когда остановить эксперимент?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/statistics/overview>

Analytics event debugger.

ОПИСАНИЕ

Локальный devtool / SaaS-tool, показывающий реальные events, filtered by user, schema validation, debug-mode (logging в console).

КАК РАБОТАЕТ

- 01 Real-time event stream.
- 02 Schema-validation per event.
- 03 Filter по user-id / property.
- 04 Export для analyst.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Product analytics dev-tools.
- 02 QA для events.

ПОДВОДНЫЕ КАМНИ

- 01 События с PII – leak.
- 02 Schema mismatch обнаруживается поздно.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое event schema?
- 02 Как validate в real-time?

ПОЛНАЯ СТАТЬЯ

<https://posthog.com/docs>

ВОПРОС Q16

Error monitoring dashboard.

ОПИСАНИЕ

Sentry-подобный internal: список ошибок (group by stacktrace), частота, affected users, releases, environment, search.

КАК РАБОТАЕТ

- 01 Group by hash(stack).
- 02 Trends graph.
- 03 Release annotations.
- 04 Filters: env, browser, version.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Internal observability.
- 02 Custom error tracking.

ПОДВОДНЫЕ КАМНИ

- 01 Cardinality explosion при динамических messages.
- 02 Без grouping – заваливает.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое error fingerprinting?
- 02 Чем Sentry от Bugsnag отличается?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/>

Frontend performance playground.

ОПИСАНИЕ

Internal tool: запускает Lighthouse в headless Chrome, показывает CWV, сравнивает версии, regression alerts.

КАК РАБОТАЕТ

- 01 Lighthouse CI / playwright.
- 02 CWV + custom metrics.
- 03 Сравнение PR vs main.
- 04 Alerts на regression.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Performance-команды.
- 02 Pre-release аудит.

ПОДВОДНЫЕ КАМНИ

- 01 Lighthouse не отражает field metrics.
- 02 Single-run – шумные данные.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CrUX?
- 02 Сколько runs нужно для стабильности?

ПОЛНАЯ СТАТЬЯ

<https://github.com/GoogleChrome/lighthouse-ci>

ДЕНЬ 6

НАГРУЗКА

ТЯЖЁЛЫЙ

5 ВОПРОСОВ

JS→TS MIGRATION**REFACTOR LEGACY****QUERY OPTIMISTIC**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

JS→TS MIGRATION

REFACTOR LEGACY

QUERY OPTIMISTIC

A11Y PACKAGE

WS CLIENT

ВОПРОСЫ ДНЯ

Q18 Migration небольшого JS-модуля на TypeScript.

Q19 Refactor legacy React component.

Q20 React Query cache + optimistic mutation.

Q21 Accessible modal/dropdown/tabs package.

Q22 WebSocket client library with reconnect/backoff.

Migration небольшого JS-модуля на TypeScript.

ОПИСАНИЕ

Пошаговая стратегия: добавить tsconfig с allowJs, .d.ts для модуля, постепенно переименовать .js → .ts, включать strict-флаги по одному.

КАК РАБОТАЕТ

- 01 allowJs+checkJs=false.
- 02 Convert один файл за раз.
- 03 Зависимости обвешать .d.ts.
- 04 Включать strict постепенно.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Legacy JS-проекты.
- 02 Adoption TS.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang миграция → утопает.
- 02 any-езде = победили компилятор.

МОГУТ ДОСПРОСИТЬ

- 01 С чего начать?
- 02 Что такое jsdoc-types?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/migrating-from-javascript.html>

ВОПРОС Q19

Refactor legacy React component.

ОПИСАНИЕ

Шаги: понять текущую логику, добавить тесты-сейфнет, разбить на меньшие компоненты, вынести логику в hooks, типизировать, удалить мёртвый код.

КАК РАБОТАЕТ

- 01 Тесты-сейфнет до изменений.
- 02 Extract custom hooks.
- 03 Split into smaller components.
- 04 Удалить unused code.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Old class-components.
- 02 Spaghetti-логика.

ПОДВОДНЫЕ КАМНИ

- 01 Refactor без тестов = регрессии.
- 02 Big rewrite вместо incremental.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое safety net?
- 02 Чем custom hooks помогают?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/workflowsOfRefactoring/>

React Query cache + optimistic mutation.

ОПИСАНИЕ

Реализация: `queryClient.setQueryData` в `onMutate`, `snapshot` для `rollback`, `invalidate` для синхронизации, тесты на `race`.

КАК РАБОТАЕТ

- 01 `onMutate`: `snapshot` + `apply`.
- 02 `onError`: `rollback`.
- 03 `onSettled`: `invalidate`.
- 04 `cancelQueries` перед `optimistic`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 `Like / favorite`.
- 02 `Inline-edit`.

ПОДВОДНЫЕ КАМНИ

- 01 Без `cancelQueries` – `race` с `refetch`.
- 02 `Snapshot` не сохранён → нечего откатывать.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `cancelQueries`?
- 02 Как тестировать `optimistic`?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates>

ВОПРОС Q21

Accessible modal/dropdown/tabs package.

ОПИСАНИЕ

Headless-пакет: focus-trap, keyboard navigation, aria-attrs, return focus, escape-to-close. Минимальные стили – composable.

КАК РАБОТАЕТ

- 01 Focus management lifecycle.
- 02 Keyboard: Esc/Tab/Arrow.
- 03 ARIA: role, aria-expanded, aria-controls.
- 04 Tested with axe + screen reader.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Внутренний UI-кит.
- 02 Open-source библиотеки.

ПОДВОДНЫЕ КАМНИ

- 01 Без focus-return – пользователь теряется.
- 02 ARIA без semantics – путает.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое headless?
- 02 Чем radix полезен?

ПОЛНАЯ СТАТЬЯ

<https://www.radix-ui.com/primitives/docs/overview/introduction>

WebSocket client library with reconnect/backoff.

ОПИСАНИЕ

Reconnect с exponential backoff + jitter, heartbeat для detect stale, queue сообщений во время disconnect, replay при reconnect.

КАК РАБОТАЕТ

- 01 Backoff: $200\text{ms} \times 2^n + \text{jitter}$.
- 02 Heartbeat ping каждые N сек.
- 03 Message queue.
- 04 onReconnect → resubscribe channels.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Real-time apps.
- 02 IoT-mqtt-bridges.

ПОДВОДНЫЕ КАМНИ

- 01 Без backoff → thundering herd.
- 02 Без heartbeat – stale connection.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое heartbeat?
- 02 Чем jitter полезен?

ПОЛНАЯ СТАТЬЯ

<https://datatracker.ietf.org/doc/html/rfc6455>

ДЕНЬ 7 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

VIRTUALIZED TABLE

MF SHELL

CI PIPELINE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

VIRTUALIZED TABLE MF SHELL CI PIPELINE

ВОПРОСЫ ДНЯ

Q23 Virtualized table with dynamic row height.

Q24 Microfrontend shell + one remote.

Q25 CI pipeline для frontend app.

Virtualized table with dynamic row height.

ОПИСАНИЕ

Variable-height rows: измерение через `ResizeObserver` или `measureRef`, кэш высот, виртуализация через `TanStack Virtual` / `react-window-dynamic`.

КАК РАБОТАЕТ

- 01 `TanStack Virtual` с `dynamicSize`.
- 02 `ResizeObserver` для измерения.
- 03 Кэш высот по `id`.
- 04 Sticky header через `position absolute`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Чаты, ленты комментариев.
- 02 Editor-документы.

ПОДВОДНЫЕ КАМНИ

- 01 Без `cache` – постоянные `measures`.
- 02 `Scroll-restore` сложнее с `dynamic`.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `dynamic` от `fixed-size` отличается?
- 02 Как `scroll-restore`?

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/virtual/latest>

В О П Р О С Q 2 4

Microfrontend shell + one remote.

ОПИСАНИЕ

Shell + один remote через Module Federation: shared React, runtime composition, ErrorBoundary, fallback UI, отдельный CI.

КАК РАБОТАЕТ

- 01 webpack 5 ModuleFederationPlugin.
- 02 shared singleton.
- 03 Lazy-load remote с Suspense+EB.
- 04 Independent CI per remote.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-team SPA.
- 02 Adoption MF.

ПОДВОДНЫЕ КАМНИ

- 01 Version skew shared.
- 02 Без EB крах shell.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое eager vs lazy share?
- 02 Как сделать local-development?

ПОЛНАЯ СТАТЬЯ

<https://module-federation.io/>

CI pipeline для frontend app.

ОПИСАНИЕ

Lint → typecheck → unit → build → bundle-size → e2e smoke → deploy. Параллелить независимые. Cache deps + build. Build once, deploy many.

КАК РАБОТАЕТ

- 01 Параллель jobs (lint || type || test).
- 02 size-limit + Lighthouse CI.
- 03 Build artifact → deploy step.
- 04 Cache: pnpm/npm + build.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой production-проект.
- 02 После аудита.

ПОДВОДНЫЕ КАМНИ

- 01 Серийный pipeline → медленно.
- 02 Без quality gate – регрессии в prod.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое build once deploy many?
- 02 Какие quality gates?

ПОЛНАЯ СТАТЬЯ

<https://docs.github.com/en/actions>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Спроектировать market watch / positions / trading dashboard (Q1, Q2, Q3)
- ☐ Спроектировать order book / notifications / admin table (Q4, Q5, Q6)
- ☐ Спроектировать portal auth / PWA offline / federated remote (Q7, Q8, Q9)
- ☐ Спроектировать mini DS / dashboard URL state / form builder (Q10, Q11, Q12)
- ☐ Спроектировать flags UI / A/B / analytics / errors / perf playground (Q13-Q17)
- ☐ Описать JS→TS migration / refactor / RQ optimistic / ally / WS lib (Q18-Q22)
- ☐ Спроектировать virtualized table / MF shell / CI pipeline (Q23, Q24, Q25)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru